LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title

CC5067NT Smart Data Discovery

60% Individual Coursework

Final Submission

Academic Semester: Spring Semester 2026

Credit: 15 credit semester long module

Student Name: Rabina Khatri

London Met ID: 24058939

College ID: np05cp4s250053

Assignment Due Date: 11/05/2026

Assignment Submission Date: 10/05/2026

Submitted To: Nikesh Regmi

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Table of Figures

Table of Tables

1. Introduction to module

Smart Data Discovery (CC5067NT) is a module that is designed to enable students to gain insight into the basic concepts and application of Data Discovery and Data Mining. Through this module, students will learn the skills required to write programs to analyze, explore and understand large datasets using modern programming tools and techniques. Students will apply these concepts using Python, a programming language along with its user-friendly libraries such as Pandas, Numpy, Matplotlib, Seaborn and Scipy which gives students the skills to manipulate and analyse complex data, which they can use for future employment in this area. Students will develop a deeper understanding of data analysis and mining and how to implement in real world scenarios using a hands-on approach. This course module delves into a variety of subjects such as understanding data, getting data ready, looking at data and analyzing statistics. Students carry out practical exercises and assignments to gain practical experience by using real data sets and extracting meaningful information from them. They practice a usable approach to data, enhancing their understanding of the concept which enable students to use data efficiently to make decisions and solve problems.

2. Introduction to Project

This project mainly focuses on network traffic analysis in order to gain a clear understanding of network flows both normal and malicious in nature. In the current digital environment, computer networks are always in a vulnerable position facing a variety of cyber-attacks. Network traffic analysis is a critical component in detecting and preventing these types of attacks before they result in significant damage (Sharafaldin, 2018). By analyzing different features of network packets such as flow duration, transfer rates, and packet size, we can identify different patterns that differentiate normal network flows from malicious network flows. This report will include a comprehensive analysis of the dataset along with data preparation, exploration, and analysis in order to further machine learning based intrusion detection.

This project also highlights the significance of data reprocessing and feature engineering that contributes to the enhancement of machine learning models for intrusion detection, with a particular focus on boosting their accuracy and efficiency. In addition, this analysis can help create intelligent security systems that can be used to identify unusual behaviors and react to potential threats in real-time.

3. Aims and Objectives

Aim

The aim of this project is to analyze the network traffic data and understand the nature of normal and attack network flows and to prepare the data for machine learning model training for intrusion detection purposes.

Objective

- To load and explore the network traffic data and understand its nature and important features.
- To clean and prepare the data by handling missing and duplicate data
- To conduct statistical analysis of the data and find correlations between variables.
- To select the most important data for analysis
- To conduct a hypothesis test to find if a significant difference exists in the flow duration of normal and attack network traffic data.
- To visualize the data and use charts and plots to understand relationships and patterns in the data.

4. Tools and Libraries Used

The following tools and libraries have been used in this project to perform data analysis, statistical testing and visualization:

Python

It is a high level and general-purpose programming language that is widely used in data science, machine learning and artificial intelligence due to its simplicity and the power of its libraries. It is a language used for data analysis and visualization (Software, 2024).

Figure 1: Python

Jupyter Notebook

It is an open-source, interactive coding environment, which is used to write and execute python code in a web browser which is used for writing, running and documenting the python code with output being displayed inline (Kluyver, 2020).

Figure 2: Jupyter Notebook

Pandas

It is a Python library used for data analysis and manipulation. It is used to load the dataset, handle missing values, check and remove duplicates, filter columns and rename label.

Figure 3: Pandas Python Library

Matplotlib

It is a Python plotting library used to create visualisations such as bar charts and pie charts to be able to explore the dataset.

Figure 4: Matplotlib Python Library

Seaborn

It is a Python data visualisation library built on top of Matplotlib and used to create the boxplot for visualising the distribution of lengths of packet (Waskom, 2021).

Figure 5: Seaborn Python Library

Scipy

It is a Python library which is used to calculate scientific computing and to perform the hypothesis test to compare mean flow duration between traffic classes (Virtanen, 2020).

Figure 6: Scipy Python Library

5. Data Understanding

Data understanding is the second phase of the CRISP-DM. At this stage, the data is gathered and explored to get acquainted with the data structure, data content and data characteristics. Categorical and continuous data are the types of data that are studied to provide the analyst with a greater understanding of the data set. This is crucial in order to prepare the data correctly and to choose the appropriate statistical methods and machine learning algorithms at the modeling stage. This initial exploration yields the analyst a good sense of the data set, aiding later on in proper analysis, result assessment, report writing, etc (ScienceDirect, 2007).

5.1. Dataset Description

A dataset is a group of data that is closely associated and is stored in a structured form for easy access and processing. Data is typically stored on a table or in a database with data in rows and columns and record in a dataset which is a row of the data and the data attribute is a column of the data.

For this project, the dataset is used in network flow data gathered from actual network environment. Each row in the dataset represents one network flow with various statistical measures describing the network packets sent during the flow. The data set consists of two types of traffic. They are BENIGN and Infiltration. BENINGN is a normal everyday traffic whereas Infiltration is an attack traffic where an intruder is trying to gain unauthorized access. The dataset is also known as the CICIDS which stands for Canadian Institute for Cybersecurity Intrusion Detection System dataset (Ring, 2019). The name of the dataset in this project is ids.csv.

```
# To understand what data type the data resources are
data_type = df.dtypes
data_type
```

```
Unnamed: 0                    int64
Destination Port              int64
Flow Duration                 int64
Total Fwd Packets             int64
Total Backward Packets        int64
                              ...
Idle Mean                   float64
Idle Std                    float64
Idle Max                      int64
Idle Min                      int64
Label                           str
Length: 80, dtype: object
```

Figure 7: Understanding data types

```
df.info()
<class 'pandas.DataFrame'>
RangeIndex: 488115 entries, 0 to 488114
Data columns (total 80 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Unnamed: 0                   488115 non-null  int64
 1   Destination Port             488115 non-null  int64
 2   Flow Duration                488115 non-null  int64
 3   Total Fwd Packets            488115 non-null  int64
 4   Total Backward Packets       488115 non-null  int64
 5   Total Length of Fwd Packets  488115 non-null  int64
 6   Total Length of Bwd Packets  488115 non-null  int64
 7   Fwd Packet Length Max        488115 non-null  int64
 8   Fwd Packet Length Min        488115 non-null  int64
 9   Fwd Packet Length Mean       488115 non-null  float64
 10  Fwd Packet Length Std        488115 non-null  float64
 11  Bwd Packet Length Max        488115 non-null  int64
 12  Bwd Packet Length Min        488115 non-null  int64
 13  Bwd Packet Length Mean       488115 non-null  float64
 14  Bwd Packet Length Std        488115 non-null  float64
 15  Flow Bytes/s                 488079 non-null  float64
 16  Flow Packets/s               488115 non-null  float64
 17  Flow IAT Mean                488115 non-null  float64
 18  Flow IAT Std                 488115 non-null  float64
 19  Flow IAT Max                 488115 non-null  int64
 20  Flow IAT Min                 488115 non-null  int64
 21  Fwd IAT Total                488115 non-null  int64
 22  Fwd IAT Mean                 488115 non-null  float64
 23  Fwd IAT Std                  488115 non-null  float64
 24  Fwd IAT Max                  488115 non-null  int64
 25  Fwd IAT Min                  488115 non-null  int64
 26  Bwd IAT Total                488115 non-null  int64
 27  Bwd IAT Mean                 488115 non-null  float64
 28  Bwd IAT Std                  488115 non-null  float64
 29  Bwd IAT Max                  488115 non-null  int64
 30  Bwd IAT Min                  488115 non-null  int64
 31  Fwd PSH Flags                488115 non-null  int64
 32  Bwd PSH Flags                488115 non-null  int64
 33  Fwd URG Flags                488115 non-null  int64
 34  Bwd URG Flags                488115 non-null  int64
 35  Fwd Header Length            488115 non-null  int64
 36  Bwd Header Length            488115 non-null  int64
 37  Fwd Packets/s                488115 non-null  float64
 38  Bwd Packets/s                488115 non-null  float64
 39  Min Packet Length            488115 non-null  int64
 40  Max Packet Length            488115 non-null  int64
 41  Packet Length Mean           488115 non-null  float64
 42  Packet Length Std            488115 non-null  float64
 43  Packet Length Variance       488115 non-null  float64
 44  FIN Flag Count               488115 non-null  int64
 45  SYN Flag Count               488115 non-null  int64
 46  RST Flag Count               488115 non-null  int64
 47  PSH Flag Count               488115 non-null  int64
 48  ACK Flag Count               488115 non-null  int64
 49  URG Flag Count               488115 non-null  int64
 50  CWE Flag Count               488115 non-null  int64
 51  ECE Flag Count               488115 non-null  int64
 52  Down/Up Ratio                488115 non-null  int64
 53  Average Packet Size          488115 non-null  float64
 54  Avg Fwd Segment Size         488115 non-null  float64
 55  Avg Bwd Segment Size         488115 non-null  float64
 56  Fwd Header Length.1          488115 non-null  int64
 57  Fwd Avg Bytes/Bulk           488115 non-null  int64
 58  Fwd Avg Packets/Bulk         488115 non-null  int64
 59  Fwd Avg Bulk Rate            488115 non-null  int64
 60  Bwd Avg Bytes/Bulk           488115 non-null  int64
 61  Bwd Avg Packets/Bulk         488115 non-null  int64
 62  Bwd Avg Bulk Rate            488115 non-null  int64
 63  Subflow Fwd Packets          488115 non-null  int64
 64  Subflow Fwd Bytes            488115 non-null  int64
 65  Subflow Bwd Packets          488115 non-null  int64
 66  Subflow Bwd Bytes            488115 non-null  int64
 67  Init_Win_bytes_forward       488115 non-null  int64
 68  Init_Win_bytes_backward      488115 non-null  int64
 69  act_data_pkt_fwd             488115 non-null  int64
 70  min_seg_size_forward         488115 non-null  int64
 71  Active Mean                  488115 non-null  float64
 72  Active Std                   488115 non-null  float64
 73  Active Max                   488115 non-null  int64
 74  Active Min                   488115 non-null  int64
 75  Idle Mean                    488115 non-null  float64
 76  Idle Std                     488115 non-null  float64
 77  Idle Max                     488115 non-null  int64
 78  Idle Min                     488115 non-null  int64
 79  Label                        488115 non-null  str
dtypes: float64(24), int64(55), str(1)
memory usage: 297.9 MB
```

Figure 8: Information of all the column of dataset

5.2. Column Description of Dataset

S.N.	Column name	Description	Data Type
1	Destination Port	The port number of the destination host. Well known ports such as port 80 which is used for HTTP and port 443 is used for HTTPs are commonly targeted by attackers.	int64
2	Flow Duration	The total duration of the flow in microseconds. Attack flows tend to have different durations compared to normal flows.	int64
3	Total Fwd Packets	The total number of packets sent from the source to the destination in the forward direction.	int64
4	Total Backward Packets	The total number of packets sent back to the source from the destination also known and response packets.	int64
5	Total Length of Fwd Packets	The total length of forward packets sent to the destination.	int64
6	Total Length of Bwd Packets	The sum in bytes of all packets sent in the backward direction.	int64
7	Fwd Packet Length Mean	The average length of packets sent in the forward direction. Unusually high or low values may indicate unusual traffic patterns.	float64
8	Bwd Packet Length Mean	The average length of packets sent in response in the backward direction.	float64

9	Flow Bytes/s	The rate of bytes per second during the flow. High values may indicate data exfiltration.	float64
10	Flow Packets/s	The rate of packets per second during the flow. Very high values are a strong indicator of a DOS and DDoS attack.	float64
11	Packet Length Mean	The average length of all packets in a flow regardless of direction.	float64
12	Packet Length Std	The standard deviation of packet lengths. High values may indicate unusual traffic patterns.	float64
13	Average Packet Size	The average size of the packets throughout the flow.	float64
14	Active Mean	The average time the flow is active before it goes idle.	float64
15	Idle Mean	The average time the flow is idle before it goes active again.	float64
16	Label	The target variable which is the type of traffic. BENIGN for normal traffic or Infiltration for attack traffic.	object(string)
17	Fwd Packet Length Max/ Min/ Mean/ Std	These are Descriptive features of Maximum, Minimum, Mean and the standard deviations of Forward packet lengths. They aid in detecting deviating traffic flow since attack tools tend to use packets of the same and/or repeated sizes.	int64/ float64
18	Bwd Packet Length Max/ Min/ Mean/ Std	These features give you statistics for your packets going back out such as the max, min, mean and the standard deviation.	int64/ float64

19	FIN Flag Count	It counts fin flags to close TCP connections. If the FIN activity is unusual, the attacks could be FIN-scans.	int64
20	SYN Flag Count	It is the number of SYN flags sent to establish TCP connections. There are typically many SYN packets In a SYN flood attack.	int64
21	RST Flag Count	It counts the number of Reset (RST) flags. If the RGT are high, it could mean port-scanning and rejected connections.	int64
22	Down/Up Ratio	This feature allows you to view the ratio of downloaded to uploaded bytes. The download traffic is typically greater during normal Web surfing versus the upload traffic, which could be associated with data theft, if it is more pronounced.	int64
23	Avg Fwd Segment Size/ Avg Bwd Segment Size	Average TCP segment size of forward and backward.	float64
24	Subflow Fwd Packets/ Subflow Fwd Bytes	These features give the average number of packets / bytes in Forward Subflows in the network communication.	int64
25	Subflow Bwd Packets/ Subflow Bwd Bytes	These features are the average packets/bytes in backwards subflows.	int64

Table 1: Column name and its Description of Dataset

6. Data Preparation

Data preparation is a term used to define a series of activities involving the collection, cleaning and organization of raw data in a manner that makes it suitable for analysis or processing. Data preparation is a critical activity in data analysis and machine learning as raw data is generally incomplete, inconsistent and unstructured. (Ring, 2019)

1. Write a python program to load data into pandas Data Frame and display last 50 records.

```
#code
# Import required libraries for data handling, visualization, and statistical testing
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

Figure 9: Python Libraries is imported successfully

```
#code
# Reads the content of 'ids.csv' CSV file and stores the value in 'df'
df = pd.read_csv('ids.csv')
# Displays the last 50 rows of the loaded dataset
df.tail(50)
```

Figure 10: Code to load CSV file and read data from csv file.

	Unnamed: 0	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	...	min_seg_size_forward	Active Mean
488065	488065	53	70591	2	2	66	194	33	33	33.000000	...	32	0.00000
488066	488066	443	5572560	8	6	386	5205	205	0	48.250000	...	20	0.00000
488067	488067	3013	49	2	0	4	0	2	2	2.000000	...	24	0.00000
488068	488068	53	122130	2	2	80	272	40	40	40.000000	...	20	0.00000
488069	488069	53	98106274	2	2	114	211	58	56	57.000000	...	20	30943.00000
488070	488070	53	506	1	1	53	113	53	53	53.000000	...	32	0.00000
488071	488071	53	3957631	2	2	120	260	79	41	60.000000	...	20	0.00000
488072	488072	9102	2	2	0	4	0	2	2	2.000000	...	24	0.00000
488073	488073	2135	111	2	2	4	12	2	2	2.000000	...	24	0.00000
488074	488074	1048	49	2	0	4	0	2	2	2.000000	...	24	0.00000
488075	488075	54240	55	1	1	6	6	6	6	6.000000	...	20	0.00000
488076	488076	1049	73	2	2	4	12	2	2	2.000000	...	24	0.00000
488077	488077	443	4	2	0	12	0	6	6	6.000000	...	20	0.00000
488078	488078	53	172	2	2	80	112	40	40	40.000000	...	20	0.00000
488079	488079	443	79063	1	2	0	0	0	0	0.000000	...	32	0.00000
488080	488080	51425	4	2	0	0	0	0	0	0.000000	...	32	0.00000
488081	488081	5120	85	2	2	4	12	2	2	2.000000	...	24	0.00000
488082	488082	39930	4	2	0	0	0	0	0	0.000000	...	32	0.00000
488083	488083	443	5871633	7	4	635	168	517	0	90.714286	...	32	150944.00000
488084	488084	10616	99	2	2	4	12	2	2	2.000000	...	24	0.00000
488085	488085	443	115338719	22	18	1317	7539	864	0	59.863636	...	20	84994.45455
488086	488086	60928	52	1	1	0	0	0	0	0.000000	...	32	0.00000
488087	488087	22	1340965	41	42	2728	6634	456	0	66.536585	...	32	0.00000
488088	488088	443	1142446	78	107	997	227837	517	0	12.782051	...	32	0.00000
488089	488089	53	67977	2	2	70	130	35	35	35.000000	...	32	0.00000
488090	488090	53	32534951	2	2	130	297	79	51	65.000000	...	32	475.00000
488091	488091	53	31361	2	2	70	198	35	35	35.000000	...	40	0.00000
488092	488092	53	228451	2	2	62	240	31	31	31.000000	...	32	0.00000
488093	488093	53	216	2	2	78	160	39	39	39.000000	...	40	0.00000
488094	488094	6302	47740	2	0	6	0	6	0	3.000000	...	20	0.00000
488095	488095	53	210	2	2	70	130	35	35	35.000000	...	20	0.00000
488096	488096	443	5322619	5	5	3471	1132	1993	6	694.200000	...	20	0.00000
488097	488097	53	118556	2	2	76	300	38	38	38.000000	...	32	0.00000
488098	488098	389	69	2	2	4	12	2	2	2.000000	...	24	0.00000
488099	488099	5405	72	2	2	4	12	2	2	2.000000	...	24	0.00000
488100	488100	53	206842	1	1	48	142	48	48	48.000000	...	20	0.00000
488101	488101	8402	49	2	0	4	0	2	2	2.000000	...	24	0.00000
488102	488102	21	72	1	2	6	12	6	6	6.000000	...	20	0.00000
488103	488103	43118	69	1	1	0	0	0	0	0.000000	...	32	0.00000
488104	488104	53	48546	2	2	70	102	35	35	35.000000	...	32	0.00000
488105	488105	49848	19	2	2	12	4	6	6	6.000000	...	20	0.00000
488106	488106	443	31143	4	1	190	0	190	0	47.500000	...	32	0.00000
488107	488107	53	51160	1	1	43	113	43	43	43.000000	...	20	0.00000
488108	488108	80	23207	2	0	12	0	6	6	6.000000	...	20	0.00000
488109	488109	80	5149971	3	1	0	0	0	0	0.000000	...	32	0.00000
488110	488110	53	238	2	2	86	228	43	43	43.000000	...	32	0.00000
488111	488111	8899	3	2	0	4	0	2	2	2.000000	...	24	0.00000
488112	488112	53	250	2	2	82	206	41	41	41.000000	...	20	0.00000
488113	488113	53	70581019	2	2	117	231	66	51	58.500000	...	32	41306.00000
488114	488114	5679	121	2	2	4	12	2	2	2.000000	...	24	0.00000

50 rows × 80 columns

Active Std	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	30943	30943	9.800000e+07	0.00000	98000000	98000000	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	150944	150944	5.720685e+06	0.00000	5720685	5720685	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
93143.49644	365777	56220	9.992959e+06	27637.53213	10000000	9910557	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	475	475	3.250000e+07	0.00000	32500000	32500000	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	Infiltration
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN
0.00000	41306	41306	7.030000e+07	0.00000	70300000	70300000	BENIGN
0.00000	0	0	0.000000e+00	0.00000	0	0	BENIGN

Figure 11: Output after loading and reading file.

Code Description:

In figure 9, different external python libraries is imported using import function which is required throughout the entire notebook. Then, the CSV file is loaded into a pandas DataFrame using **read_csv()** in figure 10, which reads the CSV file named **'ids.csv'** from the local directory and stores its contents as a pandas DataFrame called **df**. After loading the CSV file, the last 50 rows is displayed from the loaded dataset using **tail(50)** function.

The very first step and the most important step of any data analysis project is to load the data. The absence of this step means that there is no data to work with. Looking at the last rows of the dataset assists the analyst to ensure that the entire file is loaded as well as records towards the end of the file may be overlooked.

Output Description:

The last 50 rows of the dataset was shown in the output from row 488065 to 488114. Every row is a network flow and the columns are different aspects of that flow.

2. **Write a Python program to create a separate Data Frame containing only the following columns list for further analysis.**
   Columns = ['Destination Port', 'Flow Duration', 'Total Fwd Packets', 'Total Backward Packets', 'Total Length of Fwd Packets', 'Total Length of Bwd Packets', 'Fwd Packet Length Mean', 'Flow Bytes/s', 'Flow Packets/s', 'Packet Length Mean', 'Packet Length Std', 'Average Packet Size', 'Active Mean', 'Idle Mean', 'Label']

Code:

```
#code
# Select important columns required for data analysis
columns = [
    'Destination Port', 'Flow Duration', 'Total Fwd Packets',
    'Total Backward Packets', 'Total Length of Fwd Packets',
    'Total Length of Bwd Packets', 'Fwd Packet Length Mean',
    'Bwd Packet Length Mean', 'Flow Bytes/s', 'Flow Packets/s',
    'Packet Length Mean', 'Packet Length Std', 'Average Packet Size',
    'Active Mean', 'Idle Mean', 'Label'
]
# Stores these 16 columns in a new DataFrame called 'df_selected'
df_selected= df[columns].copy()
# Display first 5 rows of selected dataset
df_selected.head()
```

Figure 12: Code to create new dataframe containing 16 columns

Output:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean	Label
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.00000	0.0	BENIGN
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.00000	0.0	Infiltration
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.00000	0.0	BENIGN
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636	10000000.0	BENIGN
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.00000	0.0	BENIGN

Figure 13: Output after 16 columns stored in new Dataframe successfully.

Code Description:

The 16 columns that were provided in the question are only copied from the original Dataframe to create a new Dataframe called **df_selected**. The **copy()** method is used to ensure that df_selected is an independent copy of columns, so that future modifications made to the df_selected dataframe will not accidentally change the original df dataframe, and analyst will still have the original data to refer to. The loaded top 5 rows of the data are displayed by the **head()** function.

Output Description:

This displays only **16** important columns, so the first **5 rows** are displayed. It verifies that we have successfully created a smaller, more focused DataFrame named as df_selected that includes numeric features and our Label column.

3. **Write a Python program to check for NaN(missing values) and remove them from the newly created DataFrame.**

Code:

```
# check NaN values
print("\nCount of NaN values in each column")
print("_____________________________________")
print("Column Name              Total Count"  )
print("_____________________________________\n", df_selected.isnull().sum())

# count total NaN values in the DataFrame
total_nan = df_selected.isna().sum()
print("--------------------------")
print("Total NaN values: ", total_nan.sum())
print("--------------------------")
# remove NaN values
new_df = df_selected.dropna()

#count of nan value after removing missing values
print("\nCount of NaN values in each column after removing NaN values")
print("_____________________________________________________________")
print("Column Name                Total Count")
print("_____________________________________________________________\n", new_df.isna().sum())
```

Figure 14: Code to check and remove NaN values

Output:

```
Count of NaN values in each column
_____________________________________
Column Name                Total Count
_____________________________________
 Destination Port                 0
Flow Duration                     0
Total Fwd Packets                 0
Total Backward Packets            0
Total Length of Fwd Packets       0
Total Length of Bwd Packets       0
Fwd Packet Length Mean            0
Bwd Packet Length Mean            0
Flow Bytes/s                     36
Flow Packets/s                    0
Packet Length Mean                0
Packet Length Std                 0
Average Packet Size               0
Active Mean                       0
Idle Mean                         0
Label                             0
dtype: int64
--------------------------
Total NaN values:  36
--------------------------
```

Figure 15: Output after checking NaN1 values

```
Count of NaN values in each column after removing NaN values

_____________________________________________
Column Name                Total Count
_____________________________________________
 Destination Port             0
Flow Duration                 0
Total Fwd Packets             0
Total Backward Packets        0
Total Length of Fwd Packets   0
Total Length of Bwd Packets   0
Fwd Packet Length Mean        0
Bwd Packet Length Mean        0
Flow Bytes/s                  0
Flow Packets/s                0
Packet Length Mean            0
Packet Length Std             0
Average Packet Size           0
Active Mean                   0
Idle Mean                     0
Label                         0
dtype: int64
```

Figure 16: Output after removing NaN values

Code Description:

This code indicates that if there are missing (NaN) values in the dataset, then they will be identified, counted and removed. Firstly, the number of missing values for each column will be determined by using **isnull().sum()** function in the DataFrame **df_selected**. The results are given as a formatted output such that the user can easily analyse which column has negative values and the negative values in which column. Then, the **isna().sum()** function is applied once more in order to get the total number of missing values in the complete data set by summing all missing values along each column.

Then every row of the data that has a missing value are removed by using **dropna()** function, and after removing NaN values, the dataset is cleaned which is saved in a new dataframe named as new_df. Lastly, the code again verifies that the dataset contains no missing value by running **isna().sum()** which is used to compare with the previous count. This is a crucial stage in data reprocessing as machine learning models and data analysis techniques tend to give good but not great results if the data does not have missing or incomplete data.

Output Description:

The output displays that there are no missing values for any of the other columns except for the **Flow Bytes/s** column with **36 NaN** values. There are no missing values in all the columns after ***dropna*()** which means that the dataset is now clean.

4. Write a python program to check if duplicated records exist and find their total number.

Code:

```
#Code
# Check for duplicated rows
duplicates = new_df.duplicated()

# Count total duplicates
total_duplicates = duplicates.sum()

if total_duplicates > 0:
    print("Duplicate records exist.")
    print("Total duplicated records:", total_duplicates)
else:
    print("No duplicate records exist.")
```

Figure 17: Code to check duplicated records in dataset

Output:

```
Duplicate records exist.
Total duplicated records: 139389
```

Figure 18: Output after checking total number of duplicate records in dataset

Code Description:

This code is used to check for any duplicates in the dataset. Firstly, duplicate rows are detected in the DataFrame new_df by using duplicated() function which returns True or False for every row that if the row is duplicated or not. After that, sum() function is used to retrieve the subtotal of the number of duplicated rows in the dataset after the duplicated rows have been determined. Then it is displayed using if/else condition which means if total duplicates value is greater than 0, then it displays the message that there are duplicate rows in the dataset and gives total duplicated rows or it shows the message that there are no duplicate records in the dataset.

Eliminating duplicate data and identifying them helps to create consistency and accuracy in the dataset and render it fit to further analysis.

Output Description:

The output shows that there are **1,39,389** duplicate records in the dataset which is a lot, and some duplicate values need to be eliminated at a later stage to reduce bias in the analysis.

5. **Write a python program to show total number of columns. Also display name of all columns.**

Code:

```
#Code
print("Total number of columns:", new_df.shape[1])
print("Column names: ")
print(new_df.columns)
```

Figure 19: Code to show the number and name of all column

Output:

```
Total number of columns: 16
Column names:
Index(['Destination Port', 'Flow Duration', 'Total Fwd Packets',
       'Total Backward Packets', 'Total Length of Fwd Packets',
       'Total Length of Bwd Packets', 'Fwd Packet Length Mean',
       'Bwd Packet Length Mean', 'Flow Bytes/s', 'Flow Packets/s',
       'Packet Length Mean', 'Packet Length Std', 'Average Packet Size',
       'Active Mean', 'Idle Mean', 'Label'],
      dtype='str')
```

Figure 20: Output of displaying total number and name of columns

Code Description:

The code is used to present simple details of the structure of the dataset because the total number of columns present in the data frame new_df is determined with the help of the size of the columns

using the shape[1] function. The columns in a dataset are known as features or attributes, which could include parameters like the size of the packets, the duration of the flow, or the type of protocol in a network traffic analysis dataset. By showing the total number of columns can inform the user about the number of variables that they have to consider for analyzing.

After that, all the name of the columns in the dataset is displayed by using the columns function which allows the analyst to check and verify the features that will be available to work with the data in further data-processing and data-analysis.

Output Description:

The output shows that the new DataFrame named as **new_df** has **16** columns in which the **names** of all **16** columns are displayed which verifies that only the needed features are kept for further analysis.

6. **Rename the values in the Label column as follows:**
   BENIGN – Normal Traffic
   Infiltration – Attack

Code:

```
#Code
# Show original Label name
print("_____")
print("Original Label name")
print("_____")
print(new_df['Label'].value_counts())

# Rename the values in the Label column
new_df['Label'] = new_df['Label'].replace({
    'BENIGN': 'Normal Traffic',
    'Infiltration': 'Attack'
})

# Show updated Label values
print("_____")
print("Renamed label")
print("_____")
print(new_df['Label'].value_counts())
```

Figure 21: Code to check the values in Label column before and after renaming

Output:

```
_____________________________________
Original Label name
_____________________________________
Label
BENIGN          288548
Infiltration    199531
Name: count, dtype: int64
_____________________________________
Renamed label
_____________________________________
Label
Normal Traffic    288548
Attack            199531
Name: count, dtype: int64
```

Figure 22: Output before and after renaming values of Label column

Code Description:

The code given in above figure will look at and rename the class labels. Knowing the original class names from the output of the **value_counts** function in which one can see how many times each class label occurs in the data set. This enables the analyst to get a picture of the various categories of traffic distributed before conducting any modifications. In this case the data set includes labels like BENIGN and Infiltration, which denote benign and malicious network traffic. Original labels are valuable and crucial to the dataset and to the subsequent task of labeling the data for further analysis or for machine learning. The original Labels are then displayed and then renamed to simple terms in the Label column using the **replace()** function to make them more understandable. **Benign** will be replaced by **Normal Traffic** and **Infiltration** will be replaced by **Attack**. The edited labels are then displayed again using value_counts() to verify if the labels had been edited.

Output Description:

The number of records in the dataset before renaming was **288,548** BENIGN and **199,531** Infiltration. Following the renaming, those values were renamed to Normal Traffic and Attack for easier understanding of the labels.

7. Data Analysis

Data analysis is the procedure of examining, cleaning, converting and modelling data with the intention of finding valuable information, achieve conclusions and aiding in decision making. It deals with the use of statistical and mathematical methods to generalise and interpret the patterns and relations in a dataset (Tukey, 1977).

There are several measures that are used to summarize and understand the characteristics of a data set in descriptive statistics. **Mean** is the average value of the data set that indicates the central tendency of the data. **Median** is the middle value or the 50th percentile of the data from least to greatest which is less influenced by extreme data or outliers than the mean. **Mode** is the value that occur the most in the dataset. **Standard deviation** measures the amount of dispersion from the mean in which the larger the standard deviation the more the data points deviate from the mean. **Variance** quantifies the dispersion of the data set by taking the Deviations from the mean then squaring them. In addition, skewness and kurtosis are used to describe the shape of the data distribution. The level of asymmetry of the distribution is known as **Skewness**. Positive skewness is the data that is skewed to the right whereas the data that is skewed to the left is negative skewness. The measure of the tailedness or peakedness of the distribution is known as **Kurtosis**. Positive kurtosis is also known as leptokurtic distribution which is heavier tails and more extreme values than for a normal distribution. The lighter tails with less extreme values called negative kurtosis or platykurtic (Hossain, 2024).

1. **Write a Python program to show summary statistics of sum, mean, standard deviation, skewness, and kurtosis of any 2 chosen numeric variable.**

Code:

```
#Code
# Flow Duration statistics
print("Flow Duration")
print("-------------")
print("\nSum: ", new_df['Flow Duration'].sum())
print("\nMean:", new_df['Flow Duration'].mean())
print("\nStandard Deviation:", new_df['Flow Duration'].std())
print("\nSkewness:", new_df['Flow Duration'].skew())
print("\nKurtosis:", new_df['Flow Duration'].kurt())
print("_____")
print("_____")

# Packet Length Mean statistics
print("\nPacket Length Mean")
print("--------------------")
print("\nSum:", new_df['Packet Length Mean'].sum())
print("\nMean:", new_df['Packet Length Mean'].mean())
print("\nStandard Deviation:", new_df['Packet Length Mean'].std())
print("\nSkewness:", new_df['Packet Length Mean'].skew())
print("\nKurtosis:", new_df['Packet Length Mean'].kurt())
```

Figure 23: Code to show statistics for two selected variables

Output:

```
Flow Duration
-------------

Sum:  4364852868645

Mean: 8942922.905195676

Standard Deviation: 27481820.041125722

Skewness: 3.2508743768773116

Kurtosis: 9.246790962992351
_____
_____

Packet Length Mean
--------------------

Sum: 41757628.631677166

Mean: 85.55506102839328

Standard Deviation: 173.87246683073874

Skewness: 4.935765643058977

Kurtosis: 34.61051191591095
```

Figure 24: Output of displaying summary statistics of two selected variables

Code Description:

This code contains descriptive statistics information to compute and present for two key network traffic attributes which are Flow Duration and Packet Length Mean. The **print()** function is found throughout the code used for creating headings, labels and statistical results in an organized way. This code starts by searching a column name in the DataFrame named **Flow Duration** and uses a series of statistical functions to perform analyses on this data. Similarly, statical functions are applied on the **Packet Length Mean** column to gain insights about packet size behavior in the network traffic dataset.

There are several statistical functions called upon within this piece of code. The function **sum()** is for adding up the values of all the items in a column. The **mean()** function works out the average of the data which gives the central tendency of the feature. The **std()** function calculates the standard deviation which shows how much the values are spread around the mean. The **skew()** function is used to calculate the measure of how far from the data distribution is normal, and whether it is positive or negative skewed. The **kurt()** function can be used to determine the kurtosis, which may be used to identify extreme values or outliers in the data distribution.

Output Description:

The Flow Duration Mean is about **8.9** million microseconds with a high standard deviation which is **27481820.041125722**, and skewness is **3.25** and kurtosis is **9.24** which indicates that the data is right-skewed with heavy tails means a very long flows.

The Packet Length Mean is approximately **85.5** bytes with a high standard deviation which is **173,87246683073874**, and the skewness is **4.93** and the kurtosis is **34.6** which indicates that the data is heavily skewed to the right and contains many extreme values.

2. Write a Python program to calculate and show correlation of all numeric variables. Display top 5 correlated feature.

a) Full Correlation Matrix

Code:

```
# code
# Select only numeric columns for correlation analysis
numeric_df = new_df.select_dtypes(include=[np.number])
# Generate correlation matrix
corr_matrix = numeric_df.corr()
corr_matrix
```

Figure 25: Code to generate correlation matrix

Output:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean	Idle Mean
Destination Port	1.0000	-0.1453	-0.0257	-0.0240	0.0048	-0.0156	0.0052	-0.2173	0.0638	0.2142	-0.1185	-0.1568	-0.1277	-0.0515	-0.1093
Flow Duration	-0.1453	1.0000	0.1243	0.1008	0.0320	0.0635	0.1149	0.3510	-0.0189	-0.1134	0.3248	0.4660	0.3031	0.2047	0.6531
Total Fwd Packets	-0.0257	0.1243	1.0000	0.9498	0.3336	0.9200	0.0358	0.2101	-0.0021	-0.0205	0.2004	0.1749	0.1902	0.1371	0.0746
Total Backward Packets	-0.0240	0.1008	0.9498	1.0000	0.1467	0.9615	0.0157	0.2008	-0.0029	-0.0206	0.1866	0.1567	0.1773	0.1020	0.0661
Total Length of Fwd Packets	0.0048	0.0320	0.3336	0.1467	1.0000	0.0139	0.1276	0.0163	0.0018	-0.0065	0.0802	0.0545	0.0779	0.0588	0.0134
Total Length of Bwd Packets	-0.0156	0.0635	0.9200	0.9615	0.0139	1.0000	-0.0010	0.1945	-0.0014	-0.0121	0.1760	0.1381	0.1674	0.0782	0.0485
Fwd Packet Length Mean	0.0052	0.1149	0.0358	0.0157	0.1276	-0.0010	1.0000	0.1239	0.1550	-0.0787	0.6129	0.4627	0.6283	0.0688	0.0742
Bwd Packet Length Mean	-0.2173	0.3510	0.2101	0.2008	0.0163	0.1945	0.1239	1.0000	-0.0243	-0.1668	0.8250	0.8768	0.8094	0.1878	0.2368
Flow Bytes/s	0.0638	-0.0189	-0.0021	-0.0029	0.0018	-0.0014	0.1550	-0.0243	1.0000	0.2003	0.1279	0.0691	0.1885	-0.0067	-0.0142
Flow Packets/s	0.2142	-0.1134	-0.0205	-0.0206	-0.0065	-0.0121	-0.0787	-0.1668	0.2003	1.0000	-0.1347	-0.1457	-0.1356	-0.0403	-0.0851
Packet Length Mean	-0.1185	0.3248	0.2004	0.1866	0.0802	0.1760	0.6129	0.8250	0.1279	-0.1347	1.0000	0.8878	0.9951	0.1822	0.2152
Packet Length Std	-0.1568	0.4660	0.1749	0.1567	0.0545	0.1381	0.4627	0.8768	0.0691	-0.1457	0.8878	1.0000	0.8756	0.1921	0.2701
Average Packet Size	-0.1277	0.3031	0.1902	0.1773	0.0779	0.1674	0.6283	0.8094	0.1885	-0.1356	0.9951	0.8756	1.0000	0.1717	0.2046
Active Mean	-0.0515	0.2047	0.1371	0.1020	0.0588	0.0782	0.0688	0.1878	-0.0067	-0.0403	0.1822	0.1921	0.1717	1.0000	0.2197
Idle Mean	-0.1093	0.6531	0.0746	0.0661	0.0134	0.0485	0.0742	0.2368	-0.0142	-0.0851	0.2152	0.2701	0.2046	0.2197	1.0000

Figure 26: Output of generating full correlation matrix of all numeric variables

Code Description:

The code above in the figure (25) produced a correlation matrix between the numbers in the file. The **select_dtypes(include=[np.number])** function is used to filter only numeric columns from the DataFrame since correlation analysis can only be performed on numerical data. **numeric_df** contains the data selected.

Following that, **corr()** is used to compute the correlation of all the numeric features which is stored in **corr_matrix**. A correlation matrix is used to determine the relationship between the variables in which the closer the variables are to 1, the stronger the positive relationship between the variables whereas the closer the variables are to -1, the stronger the negative relationship between the variables, and the closer to 0, the weaker the relationship, or little to no relationship between the features.

Output Description:

The output shows the large table that displays the relationship between each of the numeric columns. The closer the value **(1 to -1)**, the stronger the relationship is, and the closer the value **(0)**, the less the relationship is.

b) Top 5 correlated feature pairs

Code:

```
# Code
# correlation matrix
corr = new_df.corr(numeric_only=True)

top_corr = corr.abs().unstack().sort_values(ascending=False)
# remove self correlations
top_corr = top_corr[top_corr != 1]
top_corr = top_corr.drop_duplicates()

print("\nTop 5 Correlated Features")
print("_____")
print("Feature 1                    Feature 2                Correlation")
print("_____")
print(top_corr.head())
```

Figure 27: Code to find top 5 correlated feature pairs

Output:

```
Top 5 Correlated Features
_____________________________________________________________________
Feature 1                   Feature 2                  Correlation
_____________________________________________________________________
Average Packet Size         Packet Length Mean            0.995068
Total Length of Bwd Packets Total Backward Packets        0.961480
Total Fwd Packets           Total Backward Packets        0.949848
                            Total Length of Bwd Packets   0.920007
Packet Length Std           Packet Length Mean            0.887792
dtype: float64
```

Figure 28: Output of finding top 5 strongest correlated feature

Code Description:

This code is used to extract the numerical features which are most correlated in the data set. The first is the calculation of a correlation matrix with just the numeric columns in the DataFrame **new_df.corr(numeric_only=True)**. Next, the **abs()** function transforms any negative correlation values into positive to enable both strong positive and negative correlations to be analysed on an equal basis. The correlation matrix is converted into a list format by using the **unstack()** function, while sorting values ascending is set to False puts the correlation values from highest to lowest.

Then, code eliminates self-correlation in which a feature is perfectly correlated with itself so to have a value of 1. The **drop_duplicates()** function is used to remove repeated correlation pairs. Lastly, the **head()** function shows the top 5 feature pairs with the greatest correlation.

Output Description:

Several of the features in this data set are very highly positively correlated as revealed by the correlation analysis. **Average Packet Size** and **Packet Length Mean** show the highest correlation **(0.99)**, which means that both features have very similar information. Likewise, there is a high correlation between Total Length of Bwd Packets and Total Backward Packets **(0.96)** and between Total Fwd Packets and Total Backward Packets **(0.95)**. Additionally, Total Fwd Packets is strongly correlated with Total Length of Bwd Packets, with the correlation being **0.92**, and Packet Length Std with Packet Length Mean is **0.89**. This implies that the same information is present in these pairs some of which can be removed without any loss of information.

8. Data Exploration

Exploratory Data Analysis (EDA) or data exploration is a standard process of comprehensively studying and visualising a dataset to obtain preliminary understanding, identify trends, identify outliers, and comprehend how variables relate to each other prior to any form of formal modelling (Tukey, 1977). Data exploration is not primarily aimed at proving or disproving a hypothesis, but a rich and intuitive understanding of what the data is and what stories it tells (Peng & Matsui, 2015).

1. **Plot a bar chart showing the frequency of each category in the Label column (with values as Normal Traffic and Attack). Determine whether the Label, as the dependent variable, is balanced or not in the dataset. Provide an analysis**.

Code:

```
# Code
# Bar chart for label frequency distribution
label_counts = new_df['Label'].value_counts()

plt.figure(figsize=(8, 5))
bars = plt.bar(
    label_counts.index,
    label_counts.values,
    color=['skyblue', 'Pink'],
    edgecolor='black',
    width=0.5
)

plt.title('Frequency of Each Category in Label Column', fontsize=14, fontweight='bold')
plt.xlabel('Label', fontsize=12, fontweight='bold')
plt.ylabel('Count', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

# Distribution table
label_counts.reset_index().rename(
    columns={'index': 'Label', 'count': 'Count'}
).style.hide(axis='index')
```

Figure 29: Code to plot bar chart for label frequency distribution and to show distribution table

Output:

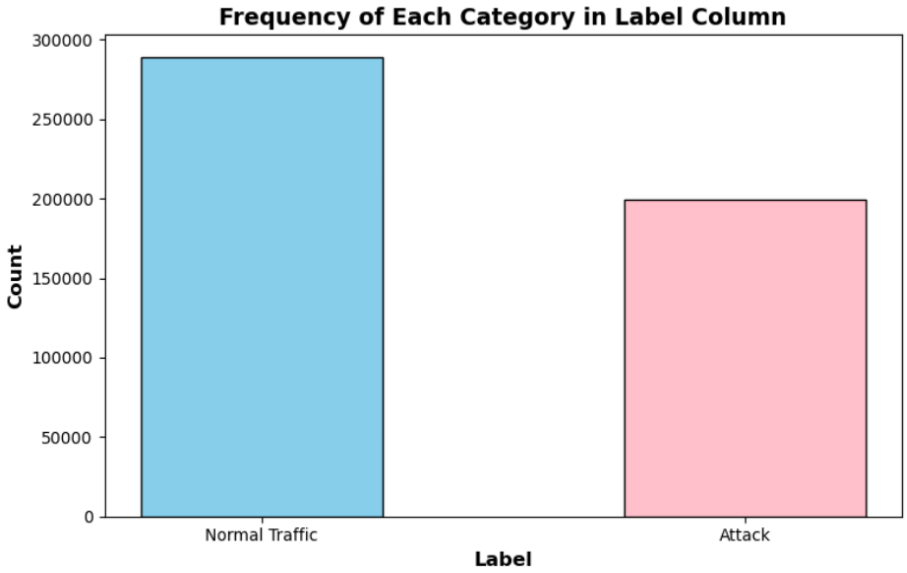

Figure 30: Output of plotting bar chart for label frequency distribution

Label	Count
Normal Traffic	288548
Attack	199531

Figure 31: Output of displaying distribution table

Code Description:

This code above in the figure (29) is used to plot and to summarise the distribution of the data in the Label column of the data. Firstly, **value_counts()** function is used to calculate the method of the number of times each unique label occurred in the data set and stores this set of numbers in **label_counts** which helps with understanding equally among different classifications, for example, normal and attack traffic. Then, the bar chart is plotted using **matplotlib.pyplot. plt.figure(figsize=( ))** sets the size of the graph, and **plt.bar( )** is used to draw a graph that will show the frequency for each label. The colors, edge color and width of the bar are customized to enhance readability **plt.title()**, **plt.xlabel()**, and **plt.ylabel()** are used to add the labels and title to the chart for better description. Finally, **plt.tight_layout()** adjusts the layout and **plt.show()** shows

the graph. Following the visualization, the code in the figure(31) also generates a table containing the same information. The **reset_index()** function changes the series to the DataFrame form and the **rename()** function changes the names of the columns to a more readable form. It is necessary to remove the index column for a tidier presentation without the use of the **style.hide(axis='index')** function. This combination allows both visual and tabular considerations of how the labels are distributed in the data.

Output Description:

The output shows that the bar chart indicates the number of Normal Traffic which is **288,548** that is significantly larger than the number of Attack which is **199,531**. The dataset is imbalanced means that there are more normal records than attack records which may have an impact on the performance of the model.

2. **Determine which category in the Label column (with values as Normal Traffic and Attack) has the highest average Flow Duration and the highest average Packet Length Mean. Visualize these findings with pie chart.**

Code:

```
# Code
# Calculate averages
avg_flow_duration = new_df.groupby('Label')['Flow Duration'].mean()
avg_packet_length = new_df.groupby('Label')['Packet Length Mean'].mean()

# Plot pie charts
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
colors = ['skyblue', 'pink']

#First pie chart
axes[0].pie(
    avg_flow_duration.values,
    labels=avg_flow_duration.index,
    autopct='%1.1f%%',
    colors=colors,
    startangle=140,
    explode=[0.05] * len(avg_flow_duration)
)
axes[0].set_title('Average Flow Duration by Label', fontsize=13, fontweight='bold')

#Second pie chart
axes[1].pie(
    avg_packet_length.values,
    labels=avg_packet_length.index,
    autopct='%1.1f%%',
    colors=colors,
    startangle=140,
    explode=[0.05] * len(avg_packet_length)
)
axes[1].set_title('Average Packet Length Mean by Label', fontsize=13, fontweight='bold')

plt.suptitle('Comparison of Average Metrics by Traffic Type', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Figure 32: Code to plot pie chart, calculate averages and show summary table of Label column

Output:

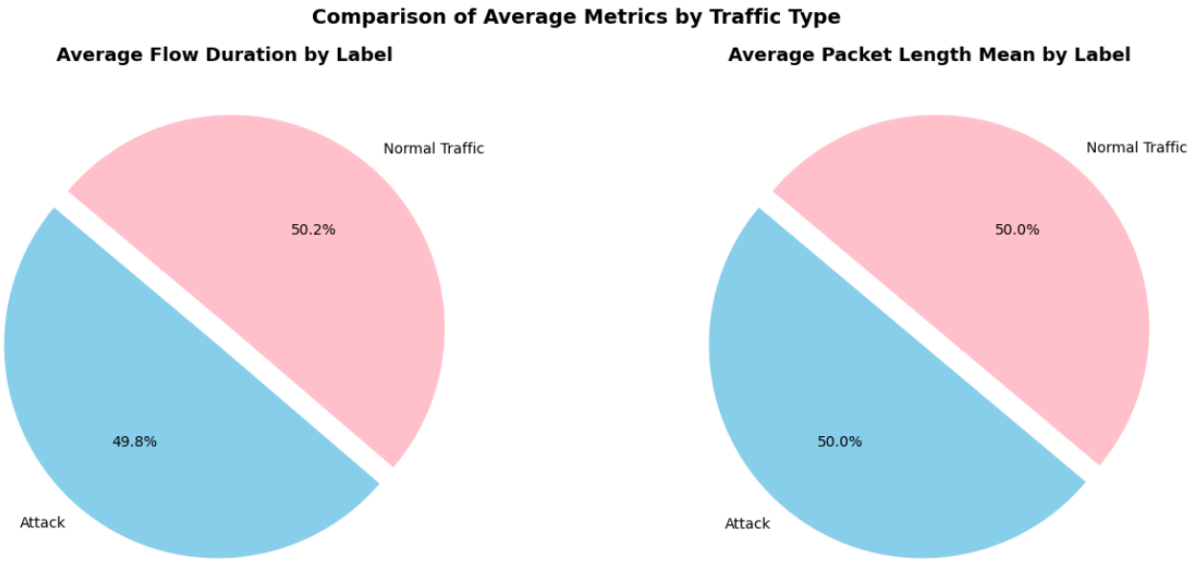

Figure 33: Output of plotting pie charts with average values for category of Label column

Code Description:

This code compares the mean value for two key network traffic attributes which are **Flow Duration** and **Packet Length Mean** for the two based on the **Label** column. The functions **groupby('Label')** and **mean()** are used to group dataset based on each class (Normal Traffic and Traffic Attack) and perform average value of each feature by group. The findings are kept in **avg_flow_duration and avg_packet_length**.

Then, two pie charts are created using matplotlib. To display both charts side by side use **plt.subplots()**. The **pie()** function graphs the relative frequency of the average values for each label and specifies the percentages using the autopct argument. An explode parameter is added, which slightly rotates away each of the slices for better chart visualization and start angle for better chart orientation. Now titles and a main heading are also added to understand it better, finally **plt.show()** is used to display the final visualization. This enables comparison of the behaviour of various types of traffic under these average network features.

Output Description:

The two pie charts compare the average value of Normal Traffic and Attack traffic as measured by the **Flow Duration** and **Packet Length Mean**. In terms of Average Flow Duration, it can be seen that both traffic types are almost equally proportioned with **49.8%**, but one category is slightly higher than the other with **50%**. In the case of Average Packet Length Mean, it can be seen that both traffic types are equally proportioned with **50%**.

3. **Generate boxplot of 'Fwd Packet Length Mean' grouped by the Label column to visualize the distribution for each attack type and normal traffic. Analyse the differences across these categories.**

Code:

```
# Code
# Boxplot for Forward Packet Length Mean grouped by label
plt.figure(figsize=(9, 6))
sns.boxplot(
    data=new_df,
    x='Label',
    y='Fwd Packet Length Mean'
)

plt.title('Boxplot of Fwd Packet Length Mean Grouped by Label', fontsize=14, fontweight='bold')
plt.xlabel('Label', fontsize=12, fontweight='bold')
plt.ylabel('Fwd Packet Length Mean', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```

Figure 34: Code to generate Boxplot for Forward Packet Length Mean grouped by label

Output:

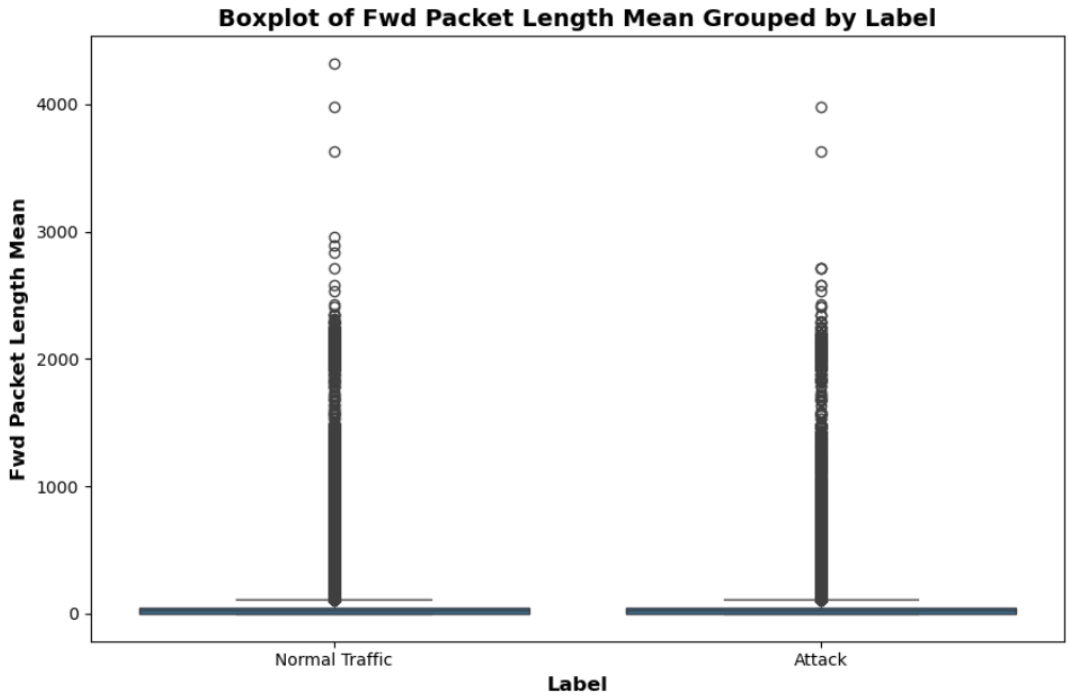

Figure 35: Output of generating BoxPlot

Code Description:

This code is used to plot **Forward Packet Length Mean** distribution for traffic categories as a **boxplot. plt.figure(figsize=(9, 6))** is used to specify the size of the graph which makes it easier to visualize. Next, the **sns.boxplot()** function in the seaborn library is used to draw the boxplot with the **x axis** as **Label** Normal and traffic attack and **y axis** as **Fwd Packet Length Mean** which helps to compare the variation of the packet sizes of various classes of network traffic.

The minimum, first quartile, median, third quartile, and maximum values and leads to the understanding that the data includes some outliers are represented by boxplot **plt.title()**, **plt.xlabel()**, and **plt.ylabel()** are the title and axis labels used to give a clear interpretation of the visualization. Finally, **plt.tight_layout()** adjusts spacing, and **plt.show()** shows off the plot.

Output Description:

The box plot of **Fwd Packet Length Mean** grouped by **Label** reveals that there are a lot of outliers in both classes, with Normal Traffic having a larger range of packet sizes and Attack having more values grouped towards the lower end of the scale that means smaller and more consistent packets are being used to attack.

4. **Perform a hypothesis test to determine if there is a significant difference in the mean Flow Duration between Normal Traffic and Attack classes. State your null and alternative hypotheses, choose an appropriate test, perform it, and interpret the results.**

Code:

```
#Code
# Null Hypothesis (H0): There is no significant difference in mean Flow Duration between Normal Traffic and Attack classes.

# Alternative Hypothesis (H1): There is a significant difference in mean Flow Duration between Normal Traffic and Attack classes.

# Normal Traffic subset
normal = new_df[new_df['Label'] == 'Normal Traffic']['Flow Duration']

# Traffic attack subset
attack = new_df[new_df['Label'] == 'Attack']['Flow Duration']

# Perform independent t-test
t_stat, p_value = stats.ttest_ind(normal, attack)

# Significance level
alpha = 0.05
print("H0: The Mean Flow Duration of Normal Traffic and Attack are equal")
print("H1: The Mean Flow Duration of Normal Traffic and Attack are different")
print("_____")
print("\nAlpha Value:", alpha)
print("T-statistic:", t_stat)
print("P-value:", p_value)
print("_____")
# Interpretation
if p_value < alpha:
    print("\nDecision: Reject H0")
    print("Conclusion: There is a significant difference in mean Flow Duration ehich supports H1.")
else:
    print("\nDecision: Fail to Reject H0")
    print("Conclusion: There is no significant difference in mean Flow Duration which holds H0.")
```

Figure 36: Code to perform hypothesis test

Output:

```
H0: The Mean Flow Duration of Normal Traffic and Attack are equal
H1: The Mean Flow Duration of Normal Traffic and Attack are different
_____

Alpha Value: 0.05
T-statistic: 0.7161327588416807
P-value: 0.4739097203854357
_____

Decision: Fail to Reject H0
Conclusion: There is no significant difference in mean Flow Duration which holds H0.
```

Figure 37: Output of performing hypothesis test with interpretation

Code Description:

The code is for hypothesis testing using statistical approach to discover if the mean Flow Duration for Attack traffic and Normal Traffic is significantly different. The **null hypothesis (H0)** is that there is no significant difference in the mean Flow Duration between the two groups and the **alternative hypothesis (H1)** is that there is a significant difference between the mean Flow Duration between two groups.

To evaluate this, the data set is also segmented based upon the Label column in the data set by using filtering in which one for the Normal Traffic, and the second for the Attack traffic. The two independent groups are then compared by means using the **ttest_ind()** function of **scipy.stats library**. This test generates two results, first is a **t-statistic** which indicates the difference between the two groups and the second results is **p-value** which indicates the probability of that difference being due to chance. The significance level (alpha) is defined to be **0.05** to run the statistical analysis.

Lastly, The p-value and alpha helps to make the decision. The Null hypothesis is rejected if the p-value is less or equal to **0.05**, and there is a statistically significant difference between Normal and Attack traffic with respect to Flow Duration. If not, null hypothesis is not rejected and there is no significant difference. This analysis is beneficial for knowing whether Flow Duration is an appropriate characteristic to help differentiate between normal and malicious network traffic.

Output Description

The output of the independent **t-test** are given below:

T-statistic = **0.716**

p-value = **0.474**

The **p-value** is more than the significance level **(α = 0.05)** so that the **null hypothesis (H0)** must be accepted which means that the difference between the Flow Duration means of Normal Traffic and Attack traffic is not statistically significant. However, Flow Duration is not a strong feature to differentiate the normal from the malicious network traffic.

9. References

Buczak, A. &. (2016). *A survey of data mining and machine learning methods for cyber security intrusion detection.* Retrieved from IEEE Communication Surveys & Tutorials: https://ieeexplore.ieee.org/document/7307098

Hossain, N. (2024, December 28). *Interpretation of Descriptive Statistics (Mean, Median, Mode, Standard Deviation, Variance, Kurtosis, and Skewness)*. Retrieved from Medium: https://medium.com/@nisma.hossain.41982/interpretation-of-descriptive-statistics-mean-median-mode-standard-deviation-variance-c31a18c36fd1

Kluyver, T. a. (2020). *a publishing format for reproducible computational workflows.* . Retrieved from Jupyter Notebooks: https://eprints.soton.ac.uk/403913/

Peng, R. D., & Matsui, E. (2015). The Art of Data Science. Leanpub. Retrieved from https://bookdown.org/rdpeng/artofdatascience/

Ring, M. e. (2019). *A survey of network-based intrusion detection data sets.* Retrieved from Computers & Security: https://www.sciencedirect.com/science/article/abs/pii/S016740481930118X?via%3Dihub

ScienceDirect. (2007). *Data Understanding*. Retrieved from ScienceDirect: https://www.sciencedirect.com/topics/computer-science/data-understanding

Sharafaldin, I. L. (2018). *Toward generating a new intrusion detection dataset and intrusion traffic characterization.* . Retrieved from SCITEPRESS DIGITAL LIBRARY: https://www.scitepress.org/Link.aspx?doi=10.5220/0006639801080116

Software, P. F. (2024). Retrieved from Python Language Reference: https://www.python.org/

Tukey, J. W. (1977). Exploratory Data Analysis. Reading, MA: Addison-Wesley.

Virtanen, P. a. (2020). *Fundamental algorithms for scientific computing in Python.* Retrieved from Nature Methods: https://www.nature.com/articles/s41592-020-0772-5

Waskom, M. S. (2021). *Statistical data visualization.* Retrieved from Journal of Open Source Software.: https://joss.theoj.org/papers/10.21105/joss.03021